LAPORAN PRATIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 4”

Disusun Oleh:
Zaki Misucha Andefi
2511533006
Dosen Pengampu **:** Dr. Wahyudi, S.T, M.T.
Asisten Praktikum : Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk tugas dalam mata kuliah Struktur Data yang merupakan lanjutan dari pembelajaran Algoritma dan Pemrograman.

Pada praktikum ini, penulis mempelajari dan mengimplementasikan struktur data Queue dan Stack menggunakan bahasa pemrograman Java. Berbagai contoh program dibuat untuk memahami konsep dasar, cara kerja, serta penerapan dari kedua struktur data tersebut dalam pengolahan data.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi materi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca dalam memahami konsep struktur data, khususnya Queue dan Stack.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
BAB II PEMBAHASAN 2
2.1 Langkah Kerja 2
2.2 Program Iterasi Queue 2
2.3 Program Queue Array 3
2.4 Program Queue Array Driver 3
2.5 Program Queue Linked List 4
2.5 Program Reverse Data 4
BAB III KESIMPULAN 5
DAFTAR PUSTAKA 6

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam dunia pemrograman yang digunakan untuk mengorganisasi dan mengelola data agar dapat diakses secara efisien. Dalam pengembangan program, pemilihan struktur data yang tepat sangat berpengaruh terhadap kinerja dan efektivitas suatu sistem.

Queue merupakan struktur data yang menggunakan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Struktur ini sering digunakan dalam berbagai kasus seperti sistem antrian, pemrosesan data, dan simulasi dunia nyata.

Selain Queue, terdapat juga struktur data Stack yang menggunakan prinsip LIFO (Last In First Out). Stack sering digunakan dalam berbagai algoritma seperti pembalikan data, parsing, dan manajemen memori. Kombinasi antara Queue dan Stack juga dapat digunakan untuk menyelesaikan permasalahan tertentu, seperti membalik urutan data.

Dalam praktikum ini, dilakukan implementasi Queue menggunakan dua pendekatan yaitu array dan LinkedList, serta penggunaan Stack untuk memanipulasi data. Dengan demikian, diharapkan mahasiswa dapat memahami konsep dan penerapan kedua struktur data tersebut secara lebih mendalam.

1.2 Tujuan

Memahami konsep Queue dan Stack serta implementasinya dalam bahasa Java, serta melatih kemampuan dalam pengolahan data menggunakan struktur data.

1.3 Manfaat

Meningkatkan pemahaman tentang struktur data, melatih logika pemrograman, serta memberikan pengalaman dalam implementasi Queue dan Stack.

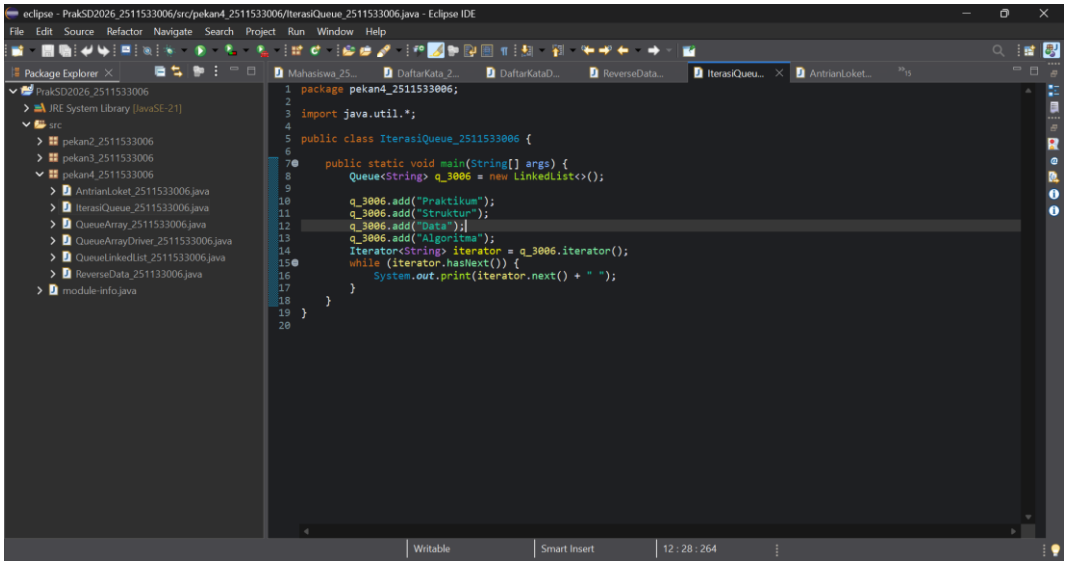
BAB II

PEMBAHASAN

2.1 Langkah Kerja

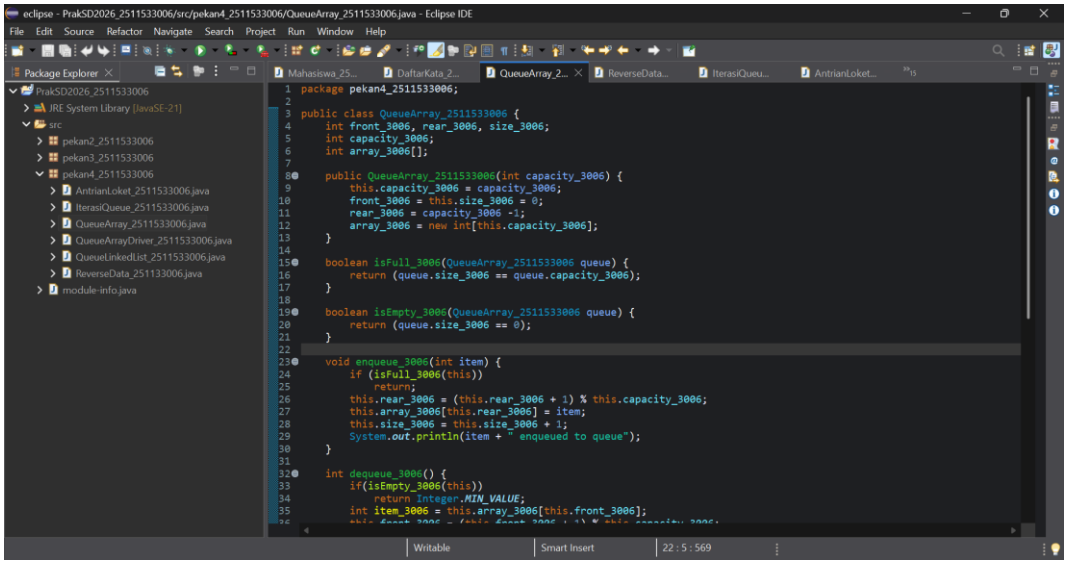
1. Membuka IDE (Eclipse).
2. Membuat package baru bernama pekan4_2511533006.
3. Menambahkan beberapa class:
    - IterasiQueue_2511533006
    - QueueArray_2511533006
    - QueueArrayDriver_2511533006
    - QueueLinkedList_2511533006
    - ReverseData_2511533006
4. Menjalankan program satu per satu dan mengamati hasilnya.

2.2 Program Iterasi Queue



Analisis: Program menggunakan Queue berbasis LinkedList dan melakukan iterasi dengan Iterator untuk menampilkan seluruh elemen tanpa menghapus data.

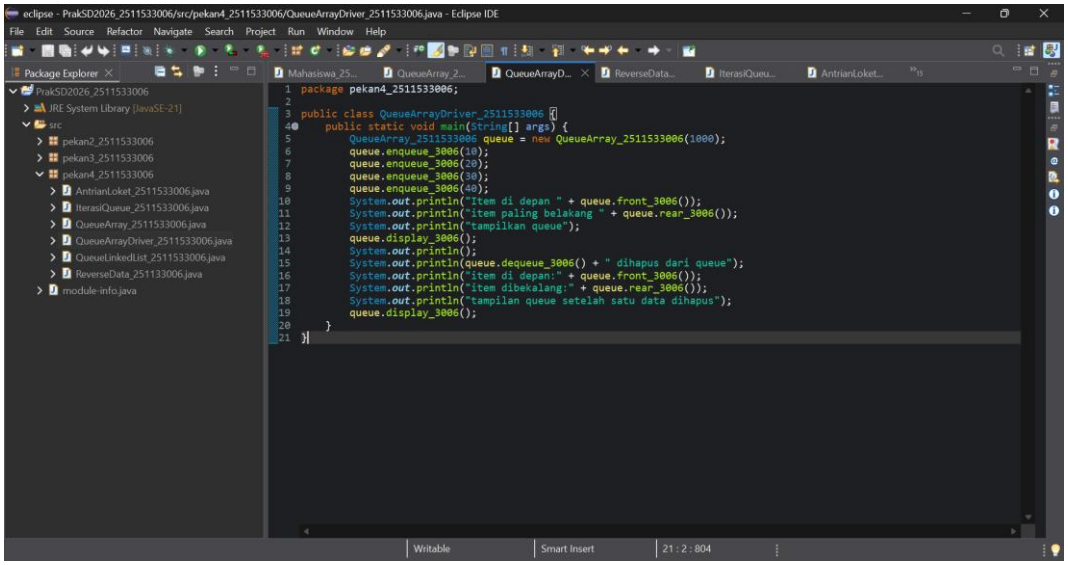
2.3 Program Queue Array



```java
package pekan4_2511533006;

public class QueueArray_2511533006 {
    int front_3006, rear_3006, size_3006;
    int capacity_3006;
    int array_3006[];

    public QueueArray_2511533006(int capacity_3006) {
        this.capacity_3006 = capacity_3006;
        front_3006 = this.size_3006 = 0;
        rear_3006 = capacity_3006 -1;
        array_3006 = new int[this.capacity_3006];
    }

    boolean isFull_3006(QueueArray_2511533006 queue) {
        return (queue.size_3006 == queue.capacity_3006);
    }

    boolean isEmpty_3006(QueueArray_2511533006 queue) {
        return (queue.size_3006 == 0);
    }

    void enqueue_3006(int item) {
        if (isFull_3006(this))
            return;
        this.rear_3006 = (this.rear_3006 + 1) % this.capacity_3006;
        this.array_3006[this.rear_3006] = item;
        this.size_3006 = this.size_3006 + 1;
        System.out.println(item + " enqueued to queue");
    }

    int dequeue_3006() {
        if(isEmpty_3006(this))
            return Integer.MIN_VALUE;
        int item_3006 = this.array_3006[this.front_3006];
```

Analisis: Implementasi Queue menggunakan array dengan konsep circular queue dengan variabel front, rear, dan size.

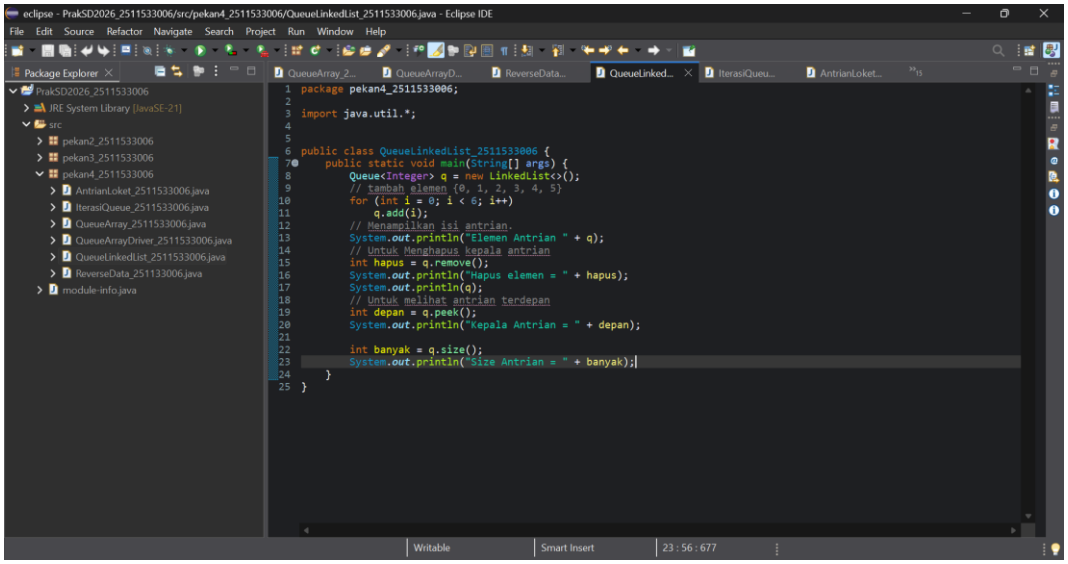
2.4 Program Queue Array Driver



```java
package pekan4_2511533006;

public class QueueArrayDriver_2511533006 {
    public static void main(String[] args) {
        QueueArray_2511533006 queue = new QueueArray_2511533006(1000);
        queue.enqueue_3006(10);
        queue.enqueue_3006(20);
        queue.enqueue_3006(30);
        queue.enqueue_3006(40);
        System.out.println("Item di depan " + queue.front_3006());
        System.out.println("item paling belakang " + queue.rear_3006());
        System.out.println("tampilkan queue");
        queue.display_3006();
        System.out.println();
        System.out.println(queue.dequeue_3006() + " dihapus dari queue");
        System.out.println("item di depan:" + queue.front_3006());
        System.out.println("item dibekalang:" + queue.rear_3006());
        System.out.println("tampilan queue setelah satu data dihapus");
        queue.display_3006();
    }
}
```

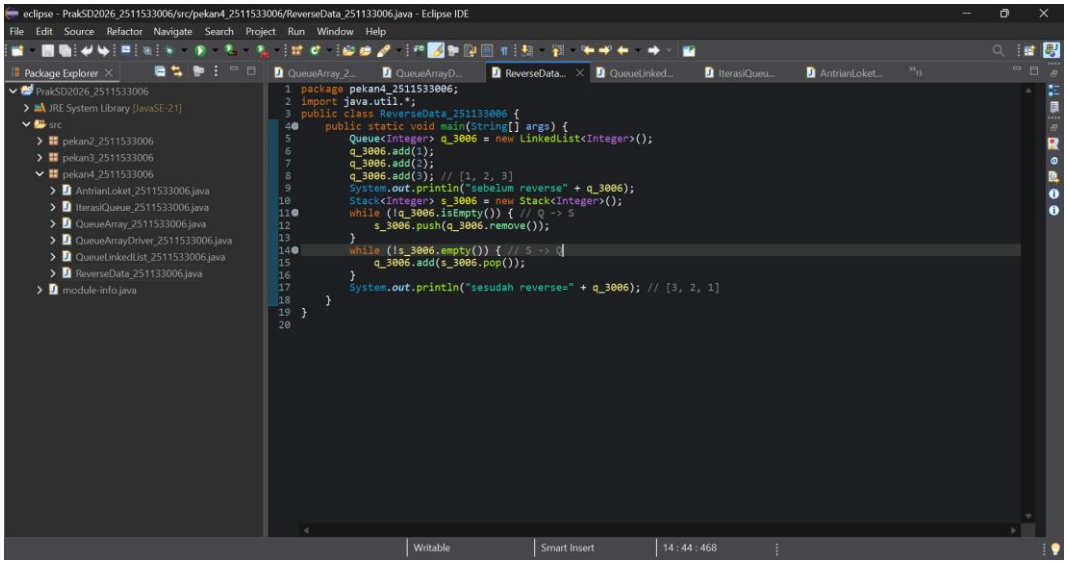
Analisis: Digunakan untuk menguji operasi QueueArray seperti enqueue dan dequeue.

2.5 Program Queue Linked List



Analisis: Menggunakan Queue bawaan Java dengan operasi add, remove, peek, dan size.

2.5 Program Reverse Data



Analisis: Menggunakan kombinasi Queue dan Stack untuk membalik urutan data.

BAB III
KESIMPULAN

Queue dan Stack merupakan dua struktur data yang sangat penting dalam dunia pemrograman karena keduanya memiliki konsep kerja yang berbeda namun saling melengkapi. Queue bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama kali masuk akan menjadi data pertama yang keluar. Konsep ini banyak digunakan dalam kehidupan nyata seperti sistem antrian, pengolahan data secara berurutan, dan manajemen tugas.

Di sisi lain, Stack menggunakan prinsip LIFO (Last In First Out), di mana data yang terakhir masuk akan menjadi data pertama yang keluar. Struktur ini sering digunakan dalam berbagai proses seperti pembalikan data, evaluasi ekspresi, serta pengelolaan pemanggilan fungsi dalam program.

Melalui praktikum ini, dapat dipahami bahwa implementasi Queue dapat dilakukan menggunakan berbagai cara, seperti menggunakan array maupun LinkedList. Selain itu, kombinasi antara Queue dan Stack juga dapat dimanfaatkan untuk menyelesaikan permasalahan tertentu, seperti membalik urutan data secara efisien.

Dengan memahami konsep dan implementasi kedua struktur data ini, mahasiswa diharapkan mampu mengaplikasikannya dalam pembuatan program yang lebih kompleks serta meningkatkan kemampuan dalam mengelola dan memproses data secara efektif dan efisien.

DAFTAR PUSTAKA

[1] H. M. Deitel dan P. J. Deitel, *Java: How to Program*, 11th ed. New Jersey, NJ: Pearson, 2017.

Tugas Struktur Data

1. AntrianLoket_2511533006

```java
package pekan4_2511533006;

import java.util.*;

public class AntrianLoket_2511533006 {
    private String[] queue;
    private int front;
    private int rear;
    private int max;

    public AntrianLoket_2511533006(int max) {
        this.max = max;
        queue = new String[max];
        front = -1;
        rear = -1;
    }

    public boolean isEmpty() {
        return front == -1;
    }

    public boolean isFull() {
        return rear == max - 1;
    }

    public void enqueue(String nama) {
        if (isFull()) {
            System.out.println("Antrian penuh! Tidak dapat menambahkan pelanggan.");
        } else {
            if (front == -1) front = 0;
            rear++;
            queue[rear] = nama;
            System.out.println("Data berhasil ditambahkan ke antrian");
        }
    }

    public void dequeue() {
        if (isEmpty()) {
            System.out.println("Antrian kosong! Tidak ada pelanggan yang dilayani.");
        } else {
            String dilayani = queue[front];
            // Geser elemen ke depan
            for (int i = front; i < rear; i++) {
                queue[i] = queue[i + 1];
            }
            queue[rear] = null;
            rear--;
            if (rear < front) {
                front = -1;
                rear = -1;
            }
            System.out.println(dilayani + " telah dilayani");
        }
```

```java
    }

    public void display() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
        } else {
            System.out.println("Isi antrian:");
            for (int i = front; i <= rear; i++) {
                System.out.println((i - front + 1) + ". " + queue[i]);
            }
        }
    }

    public void Reverse() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }
        int kiri = front;
        int kanan = rear;
        while (kiri < kanan) {
            String temp = queue[kiri];
            queue[kiri] = queue[kanan];
            queue[kanan] = temp;
            kiri++;
            kanan--;
        }
        System.out.println("Isi antrian:");
        for (int i = front; i <= rear; i++) {
            System.out.println((i - front + 1) + ". " + queue[i]);
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        AntrianLoket_2511533006 antrian = new AntrianLoket_2511533006(10);
        int pilihan;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = scanner.nextInt();
            scanner.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama = scanner.nextLine();
                    antrian.enqueue(nama);
                    break;
                case 2:
                    antrian.dequeue();
                    break;
```

```
            case 3:
                antrian.display();
                break;
            case 4:
                antrian.Reverse();
                break;
            case 5:
                System.out.println("Program selesai");
                break;
            default:
                System.out.println("Pilihan tidak valid!");
        }
    } while (pilihan != 5);

    scanner.close();
  }
}
```